



16. Density-Based Clustering

Where Dense Regions Become Clusters

Previous Section:

 [15. K-Means Clustering](#)

Contents:

[1. DBSCAN – What makes it different from K-Means?](#)

[2. DBSCAN – Why and where it stands out?](#)

[3. DBSCAN – The Workflow](#)

[4. DBSCAN – The Epsilon \(\$\epsilon\$ \)](#)

[5. DBSCAN with Python](#)

[Summary](#)

[References](#)

This section covers **Density-Based Clustering**, or just **DBSCAN** as people like to call it, another unsupervised learning model... but one that thinks very differently from what we've seen so far.

If K-Means looks at the world through **distance**, then DBSCAN sees something else entirely: **Density**.

And density, in the simplest sense, means this: *"How crowded is a region of data?"*. Are points tightly packed together, or scattered and isolated? That small shift in perspective changes everything.

Because instead of forcing data into a fixed number of clusters, DBSCAN does something much more natural. It looks at the dataset and asks:

“Where are the dense regions... and where is just noise?”

And from that, clusters begin to **emerge on their own**. That’s the beauty of DBSCAN. You don’t tell it how many clusters to create. You don’t guide it into splitting the data evenly. It simply observes and says:

- “This region is dense → that’s a cluster”
- “This region is sparse → that’s probably noise”

So unlike K-Means, which tries to divide everything into groups no matter what. DBSCAN is selective. It doesn’t just create clusters—it also has the courage to say: **“Some points... don’t belong anywhere.”**

And that’s something incredibly powerful. Because in real-world data, not everything fits nicely into a group. Some data points are just outliers, noises, exceptions. DBSCAN embraces that reality. Instead of forcing structure onto the data, it **discovers structure from density**.

1. DBSCAN – What makes it different from K-Means?

DBSCAN, or by its full name, **Density-Based Spatial Clustering of Applications with Noise**, is one of those algorithms that sounds complicated at first, but once you understand its perspective, everything becomes surprisingly clear.

It is an unsupervised learning model, just like K-Means, but it approaches the idea of clustering in a completely different way.

- Instead of asking like K-Means: “Which points are closest to this center?”
- DBSCAN asks something much more natural: “Which points are crowded together... and which ones are alone?”

At its core, DBSCAN is designed to divide a dataset into **meaningful groups based on density**. Not just distance, not just position. But how tightly data points are packed in a region.

- If a group of points is close enough and dense enough → it becomes a cluster.
- If a point is too isolated → it becomes something else entirely → Noise.

And this is where DBSCAN becomes really interesting. Because unlike many traditional clustering algorithms, it doesn’t try to force every data point into a group. It accepts that some points simply don’t belong anywhere. These are

outliers. And DBSCAN doesn't ignore them. It **detects them automatically**. That makes it incredibly useful, not just for clustering, but also for **data cleaning and anomaly detection**.

So how does it actually see the data? Think of your dataset as a map. Some regions are crowded—points packed closely together. Other regions are empty, or scattered. DBSCAN scans this space and builds clusters like this:

- Dense regions → become clusters
- Sparse regions → act as boundaries
- Isolated points → labeled as noise

In other words, clusters are not defined by shape or count. They are defined by **continuity of density**. If you can “walk” from one point to another through nearby neighbors, staying within dense regions, then they belong to the same cluster. If you can't, that point is on its own.

And this leads to one of DBSCAN's biggest strengths.

- It doesn't assume clusters are circular.
- It doesn't assume clusters are equal in size.
- It doesn't assume anything about shape at all.

Clusters can be long, curved, irregular, even weirdly connected. And DBSCAN will still find them. Because it is not trying to *fit* the data into a structure. It is trying to **discover the structure that already exists**.

So, what makes DBSCAN different from K-Means? The difference is almost philosophical.

- **K-Means:**
 - Needs you to specify the number of clusters (K)
 - Uses centroids (centers)
 - Assigns every point to a cluster
 - Works best with clean, well-separated, spherical data
- **DBSCAN:**
 - Does *not* require the number of clusters
 - Has no concept of centroids
 - Can label points as noise

- Works well with irregular, real-world data

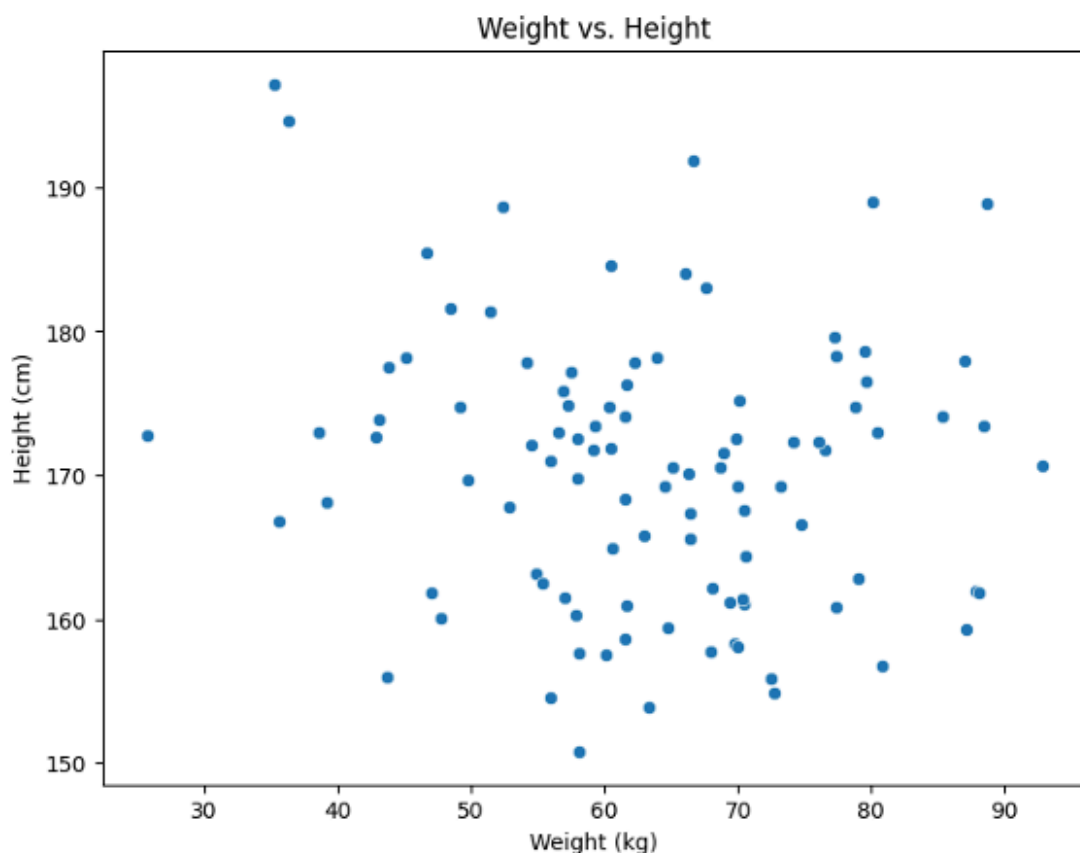
So while K-Means tries to **organize** the data, DBSCAN tries to **understand** it. And sometimes, that difference is everything.

2. DBSCAN – Why and where it stands out?

Let's make this a bit more visual.

Imagine a simple graph: X-axis → **Weight**; Y-axis → **Height**

At the beginning, nothing is labeled. No clusters. Just points.



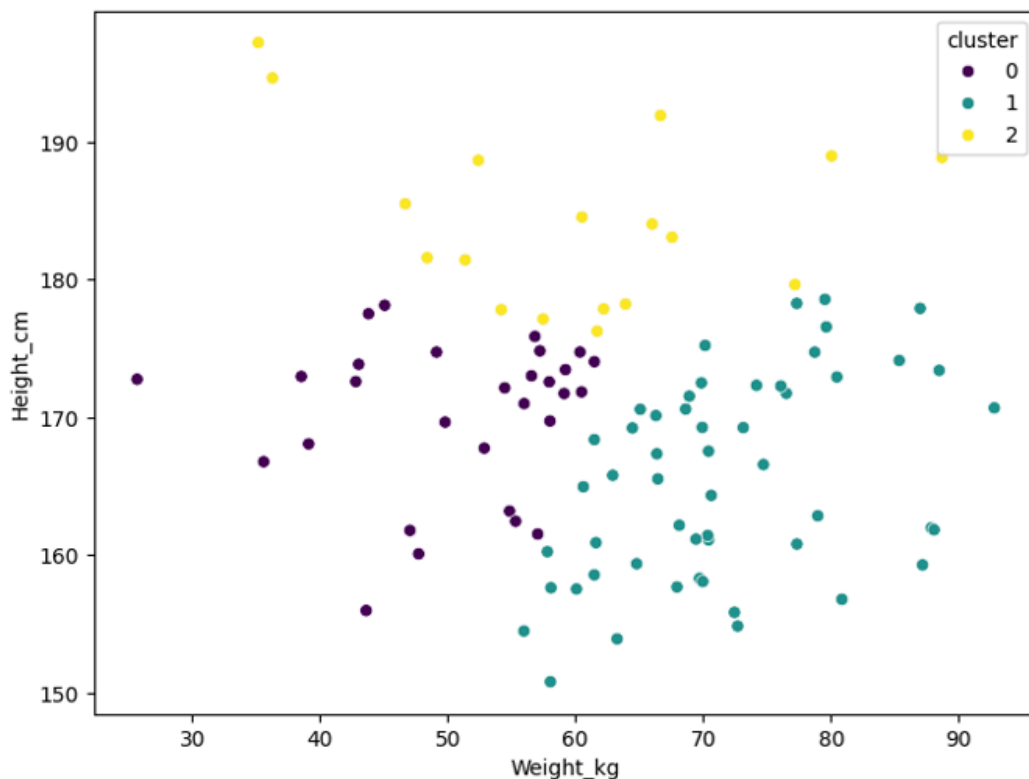
But the moment you look at it, something interesting happens. You don't need an algorithm to tell you this—your brain already sees it: There are **two natural groups**. And at the same time, there are a few points that feel off. They don't belong to either group. They are too far. Too isolated

→ Those are **outliers**. Or in DBSCAN's language: **Noise**.

And this right here is where DBSCAN starts to shine.

The Problem with Traditional Clustering (K-Means)

Now let's take that same dataset, and hand it to K-Means.



At first, it feels like it should work fine. After all, there are clearly two clusters, right? Here I decided to have three clusters instead. At the end of the day, you still need to define the number of clusters when it comes to K-Means.

But then, things start to go wrong. There's a classic issue here called **nesting**. This is when one cluster wraps around another, like a ring surrounding a core. Visually, it makes perfect sense to us. But for K-Means, this is a nightmare.

Why? Because K-Means doesn't actually "see" shapes. It only sees **distance to centroids**. So what does it do? It places centers (centroids) → Measures distance → Assigns every point to the nearest one

That's it. No understanding of structure. No awareness of density. Just distance. And because of that, it comes with some strong assumptions:

- Clusters are **spherical**

- Clusters are **separate and non-overlapping**
- Every point **must belong** to a cluster
- Outliers are **forced somewhere**, even if they don't fit

So in cases like this, some points that *clearly belong to one cluster visually* end up being assigned to another. Not because it makes sense. But because the centroid is closer.

And that's the limitation. K-Means tries to **impose structure** on the data. Even when that structure doesn't exist.

How DBSCAN Solves This: DBSCAN walks in and asks a completely different question. Not: "Which center is closest?". But: *"Is this region dense enough to be a cluster?"*

And to answer that, it uses two simple ideas:

- ϵ (**epsilon**) → How close do points need to be to be considered neighbors?
- *MinPts* → How many neighbors are enough to call something "dense"?

That's it. From just these two parameters, everything emerges.

Now DBSCAN looks at each point and classifies it:

- **Core point** → surrounded by enough neighbors → a strong signal of a cluster
- **Border point** → close to a cluster, but not dense enough itself
- **Noise point** → isolated, belongs nowhere

And then something beautiful happens. Clusters are not built from centers. They are built by **connecting dense regions together**. If you can move from one point to another, step by step, through nearby neighbors. Then they belong to the same cluster.

Why This Changes Everything? Because now:

- Clusters can be **any shape** (curved, stretched, weird... doesn't matter)
- Nested clusters are handled **naturally**
- Outliers are **recognized**, not forced
- Structure is **discovered**, not imposed

So going back to our Weight–Height example. DBSCAN does exactly what your eyes did instantly:

- It groups the dense regions into clusters
- It leaves the strange, isolated points alone

No forcing. No guessing. Just understanding.

K-Means is simple. Fast. Efficient. But it works best in a clean, ideal world.

DBSCAN, on the other hand, is built for reality. → Messy data. Irregular shapes. Hidden structures.

Because instead of asking: “Where should this point go?”

It asks something deeper: “**Does this point even belong anywhere?**”

3. DBSCAN – The Workflow

DBSCAN works in a way that feels almost organic. It doesn't try to draw lines or place centers. It simply looks at the data, and lets clusters **grow from density**.

At a high level, here's what it does:

- It can **handle nested clusters** without confusion
- It **expands clusters outward** from dense regions
- It completely **ignores geometric shape** (no circles, no assumptions)
- It identifies three types of points: **core, border, and noise**
- And it scales naturally to higher dimensions, even in 3D, 4D, or more

Everything begins the same way: raw, unclustered data. No labels. No structure. Then DBSCAN looks for one thing: **Density**.

- Where points are crowded → clusters form.
- Where points are sparse → they fade into noise.

Steps in the DBSCAN Algorithm

Now let's walk through how it actually works, step by step.

1. Identify Core Points: DBSCAN starts by examining each point, one by one.

- For every point, it asks: "How many neighbors are within a small radius ϵ ?". If the number of nearby points is **greater than or equal to MinPts**, then that point is considered a **core point**.
- This is the algorithm's way of saying: "This area is dense enough to matter."

2. Form Clusters: Once a core point is found, something interesting happens.

- If it hasn't been assigned to any cluster yet, DBSCAN creates a **new cluster** starting from that point.
- But it doesn't stop there. It begins to **expand outward**. It looks at all points within the ϵ radius, then from those points, it keeps expanding further.
- Like a ripple spreading across water. Any point that is **density-reachable** gets pulled into the cluster.

3. Density Connectivity: Now here's the subtle part, and also the powerful one.

- Two points don't need to be directly close, they just need to be **connected through density**.
- If you can form a chain like this: ***Point A*** $\rightarrow$ ***Point B*** $\rightarrow$ ***Point C*** $\rightarrow$... Where each step stays within ϵ distance, and at least one point in that chain is a core point. Then all of them belong to the same cluster.
- This idea of **density connectivity** is what allows DBSCAN to capture complex shapes, handle curved or stretched clusters, and avoid breaking clusters apart unnecessarily
- It's not about direct distance anymore. It's about whether you can **walk through dense regions** to get there.

4. Label Noise Points: Finally, after all expansions are done. Some points are left behind. They were never reached. Never part of any dense region.

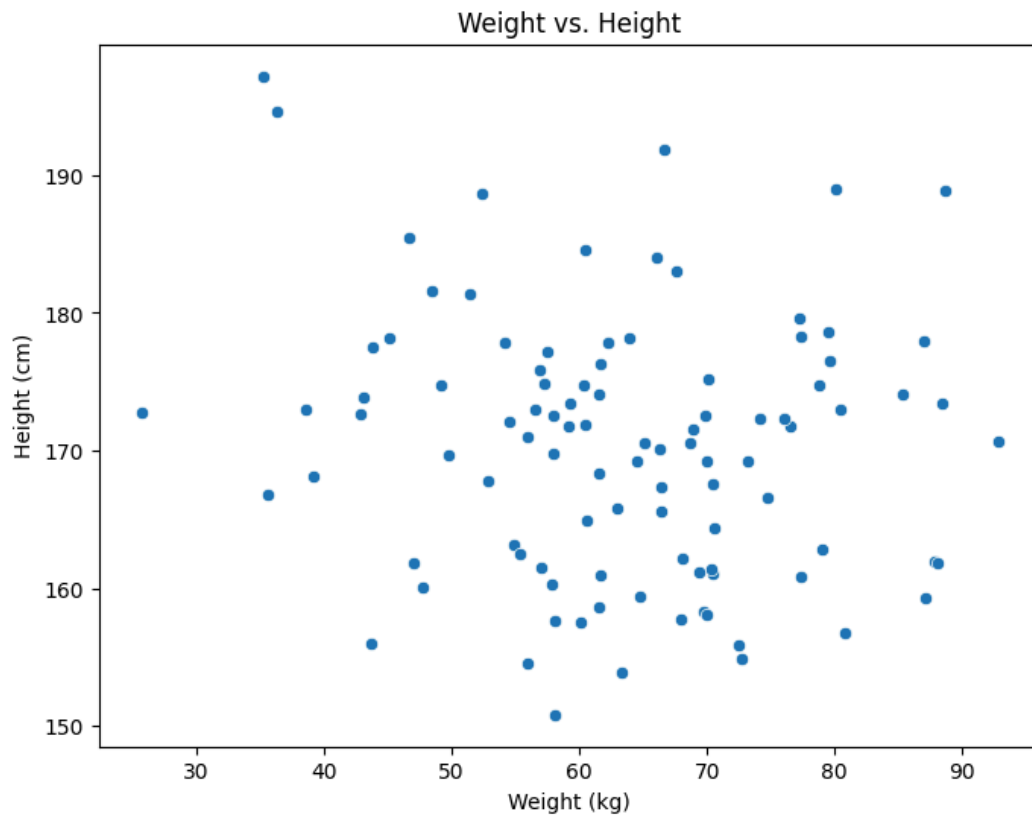
- DBSCAN looks at them and simply says: "These don't belong." $\rightarrow$ And labels them as **noise**.

So if you step back and look at the whole process... DBSCAN doesn't build clusters from the top down. It builds them from the inside out:

Find density $\rightarrow$ Grow clusters $\rightarrow$ Connect regions $\rightarrow$ Ignore what doesn't fit

No assumptions. No forced assignments. Just the structure emerging naturally from the data.

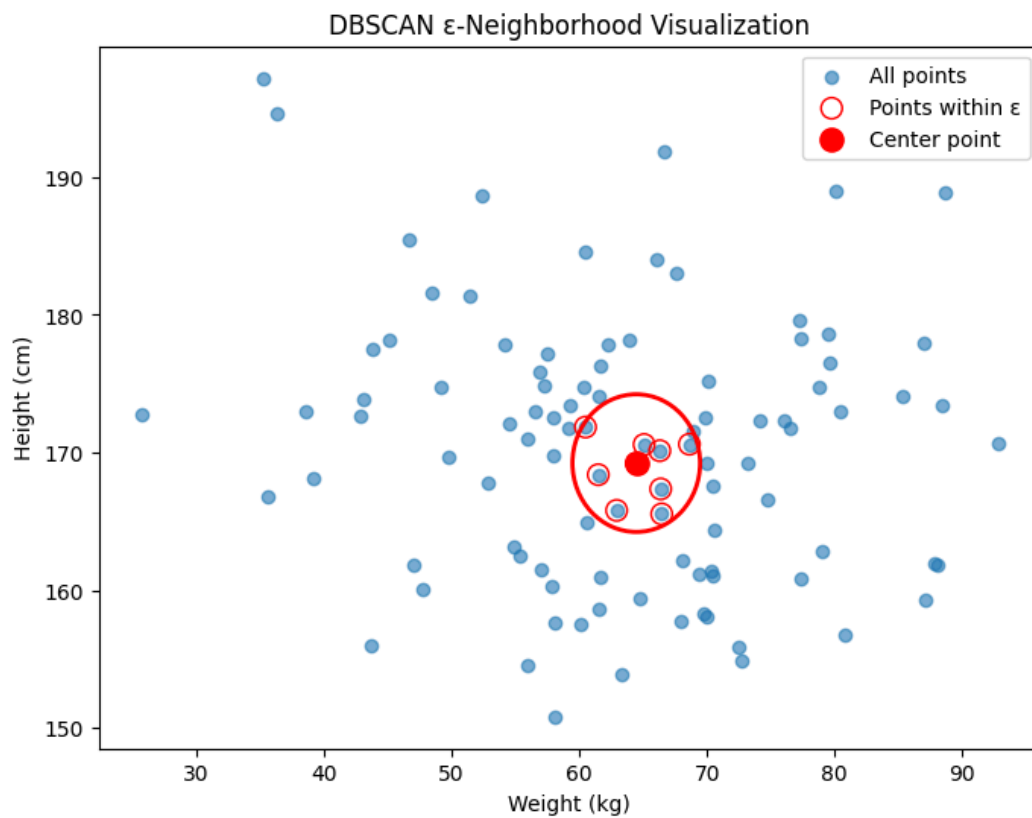
Now, let's go back to the very beginning:



No clusters. No labels. Just raw data points scattered across the space.

In DBSCAN, everything starts with a simple idea: Imagine drawing a **circle around each point**. This circle has a fixed radius, called ε (**epsilon**). So when we say $\varepsilon = 5$, what we really mean is:

"For every point, look at everything within a radius of 5". → That circle defines its **neighborhood**.



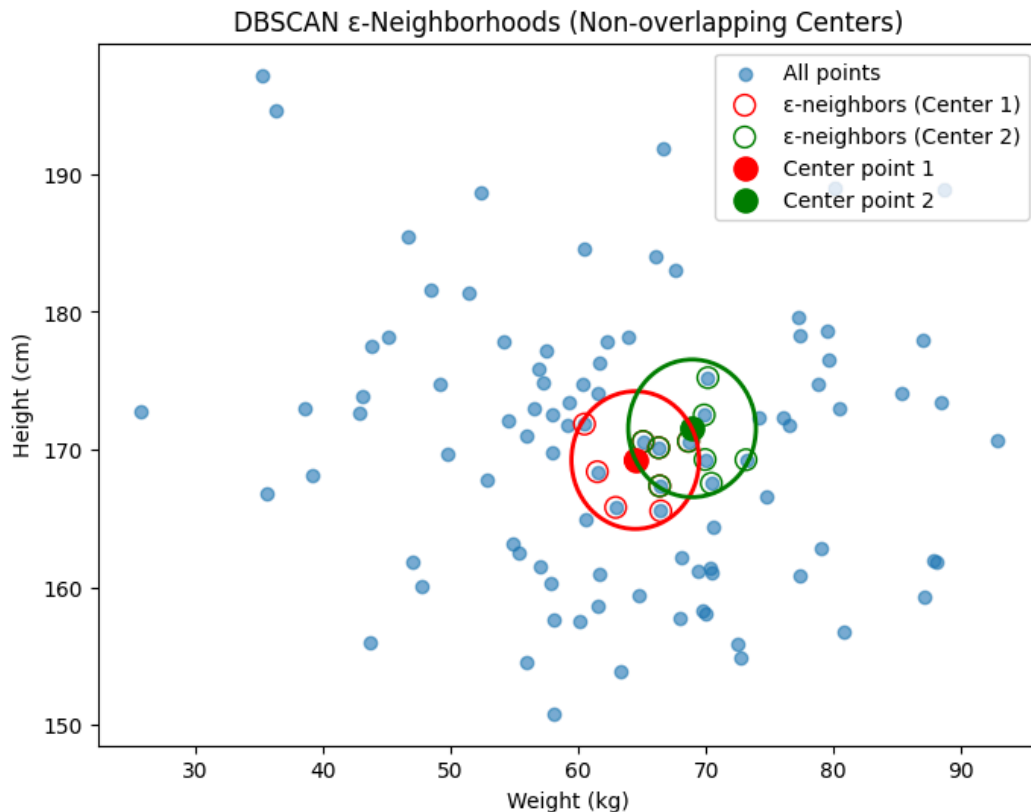
The red circle overlapping 8 points (excluding the center point)

Now picture this: You pick one point, place it at the center, and draw a circle around it.

Inside that circle, you count how many points exist.

- Points inside or on the boundary → **neighbors**
- Points outside → ignored

And here's the important part: Every single point in the dataset gets its **own circle**. Same radius. Different results. Because the data is not evenly distributed.



The green circle overlapping 9 points (excluding the center point)

Some circles will be crowded... Others will feel almost empty. And that difference... is everything.

Density in Action

- If a point sits in a dense region. Its circle will contain many neighbors.
- If a point is isolated. Its circle will contain very few, or none at all.

This variation is exactly what DBSCAN uses to understand the data.

Let's make it concrete. Suppose: $\varepsilon = 5$ and $MinPts = 9$

Now you examine a point (like what I have showcased so far → Look at the above figures alongside reading this part):

- Inside its circle, you find **8 other points**. Including itself, that makes **9 points total**. That point is now classified as a **core point**. Because it satisfies the condition: *"Dense enough to form a cluster."*

But What About Its Neighbors?

Now things get more interesting. Those 8 neighboring points are not automatically the same type. Each one has its **own circle**. Its own

neighborhood. Its own story. So they can become:

- **Core points** → if *they also* have enough neighbors (8 in this case)
- **Border points** → if they don't meet *MinPts*, but lie near a core point
- **Noise points** → if they are too isolated and not connected to any core

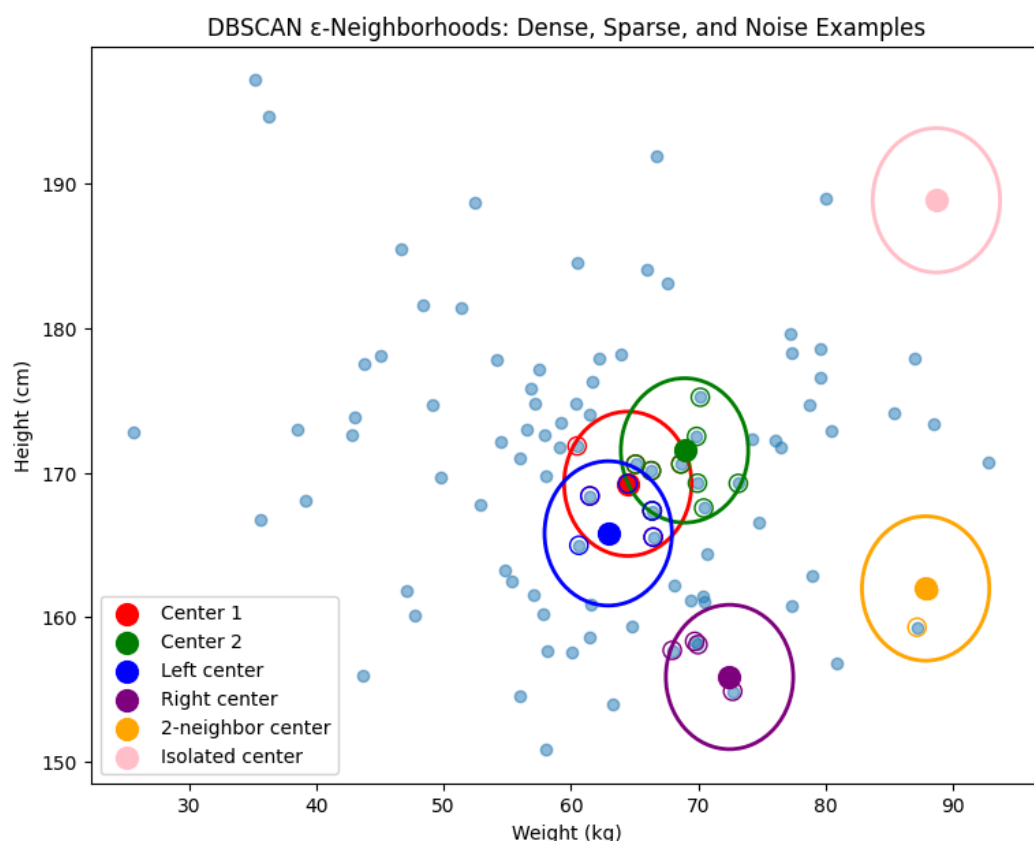
And this is a subtle, but powerful idea. A point is not judged only by itself. It is judged by **its own density** and **its relationship to nearby core points**.

Sparse Regions → Now imagine points in less dense areas.

- A point with only 4 neighbors → not dense enough
- Another with just 1 neighbor → even weaker
- A point with no neighbors at all → completely isolated

These points don't qualify as core points. Some might still be saved if they are close to a core point → **border points**.

But others? They are simply labeled as **noise**.



The blue point is close to 4 points, the purple point is close to 4 points, the yellow point is close to 1 point, the pink point has no neighbor points

So DBSCAN repeats this process... again and again... for every point:

Draw a circle → Count neighbors → Classify the point → Expand clusters from core points

And slowly, something emerges. Clusters are not assigned. They are **revealed**. Dense regions connect. Sparse regions break apart. Outliers fade away.

And in the end, what you get is not an artificial grouping. But something much closer to how we naturally see the data: ***Structure shaped by density.***

4. DBSCAN – The Epsilon (ϵ)

If there is one parameter you must understand in DBSCAN. It's this one.

Epsilon (ϵ) is the *heart* of DBSCAN. Much like how K controls the outcome in K-Means. In DBSCAN, ϵ controls how the algorithm sees the data.

Think of ϵ as a simple question: ***"How far should we look... to decide if points are neighbors?"***

That distance becomes the **radius of every circle** we draw around each point. And from that one decision, everything else follows.

For every point, DBSCAN draws a circle of radius ϵ and counts how many points fall inside.

Now combine that with another parameter: *MinPts* is the minimum number of points required to call a region "dense". And suddenly, we can classify every point.

Three Types of Points - A point becomes one of three things:

Type	Condition
Core point	Has $\geq MinPts$ points in its ϵ -neighborhood
Border point	Has $< MinPts$, but is close to a core point
Noise point	Has $< MinPts$, and is not close to any core point

- A **core point** is strong. It lives in a dense region. It can **start a cluster**, or **expand one**.
- A **border point** is weaker. It doesn't have enough neighbors on its own... But it is close enough to a core point to be *pulled into the cluster*.

- A **noise point**? It stands alone. Too far from everything. No dense region connects to it. So DBSCAN simply says: "This doesn't belong."

Let's say: $MinPts = 4$. Now we examine the dataset.

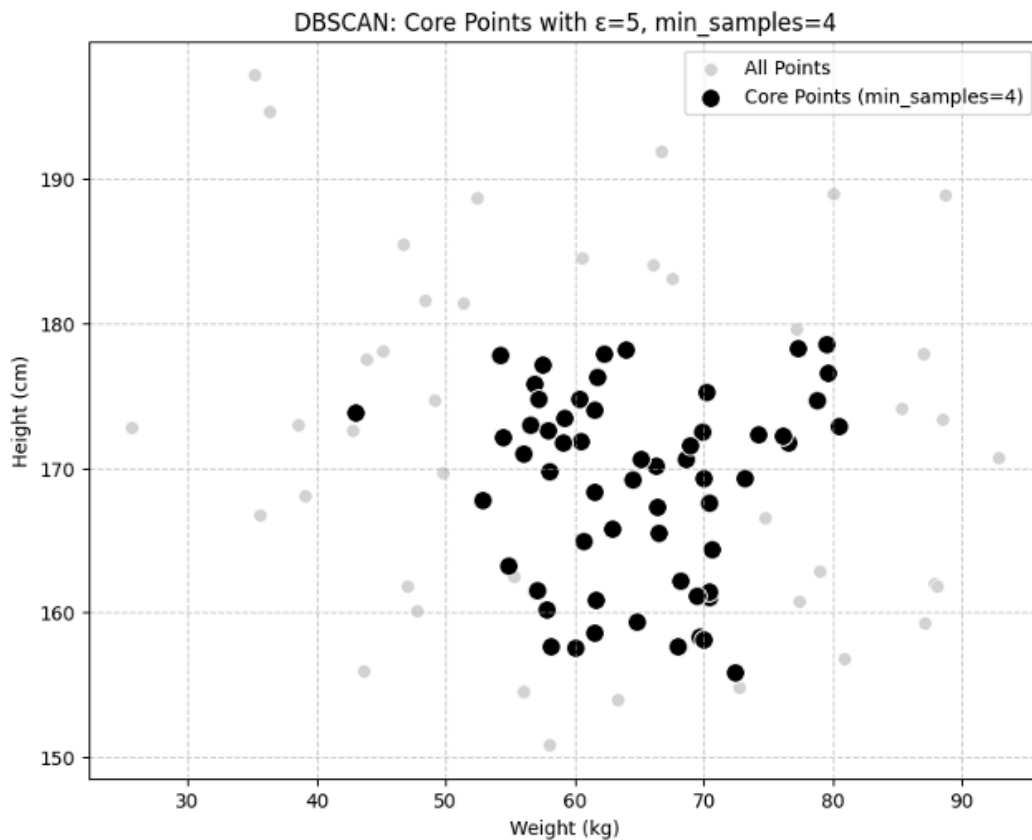
- Any point with **4 or more neighbors inside its ϵ -circle** → becomes a **core point**
- In this example:
 - 4 points satisfy this condition → **core points**
 - 2 points do not → they are *not core points*
- Now comes the important question: What are those 2 points? We don't decide immediately. We check their relationship to the core points.
 - If they lie within ϵ distance of a core point → they become **border points**
 - If they are not close to any core point → they become **noise**

Why ϵ Matters So Much: Because ϵ controls **everything**:

- Too small → almost no neighbors → everything becomes noise
- Too large → everything becomes connected → one giant cluster

So choosing ϵ is not just a parameter tuning step. It is deciding: **What does "dense" mean in this dataset?**

And that's why we say: K defines K-Means. But ϵ defines DBSCAN. Because once ϵ is set. The clusters don't get assigned. They **emerge**.



All core points determined after evaluating all the points

Now, after evaluating every point in the dataset. We finally start to see something take shape.

- Some points clearly satisfy the density condition → these are **core points**.
- The rest? They are simply **non-core points**.

But here's something important. Being a non-core point does **not** mean it gets excluded. Not yet.

Because in DBSCAN, belonging is not just about yourself. It's about your **connection to dense regions**.

Core vs Non-Core is not the final decision - Because a non-core point still has two possible paths:

- If it is close enough to a core point → it becomes a **border point**
- If it is not connected to any core point → it becomes **noise**

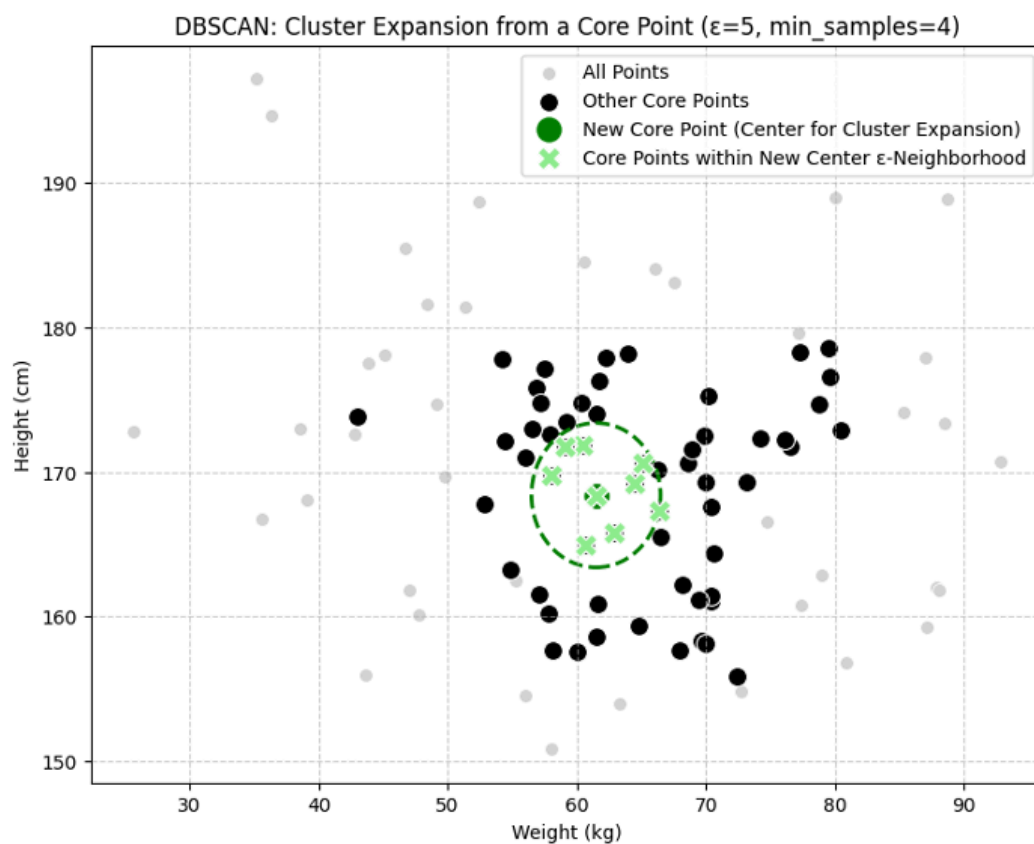
So the real question is not: "Are you dense?". But rather: "Are you connected to density?"

The Most Critical Step: Forming Clusters

Now comes the heart of DBSCAN. **Cluster assignment.** And it begins very simply.

- **Step 1: Start with a Core Point**

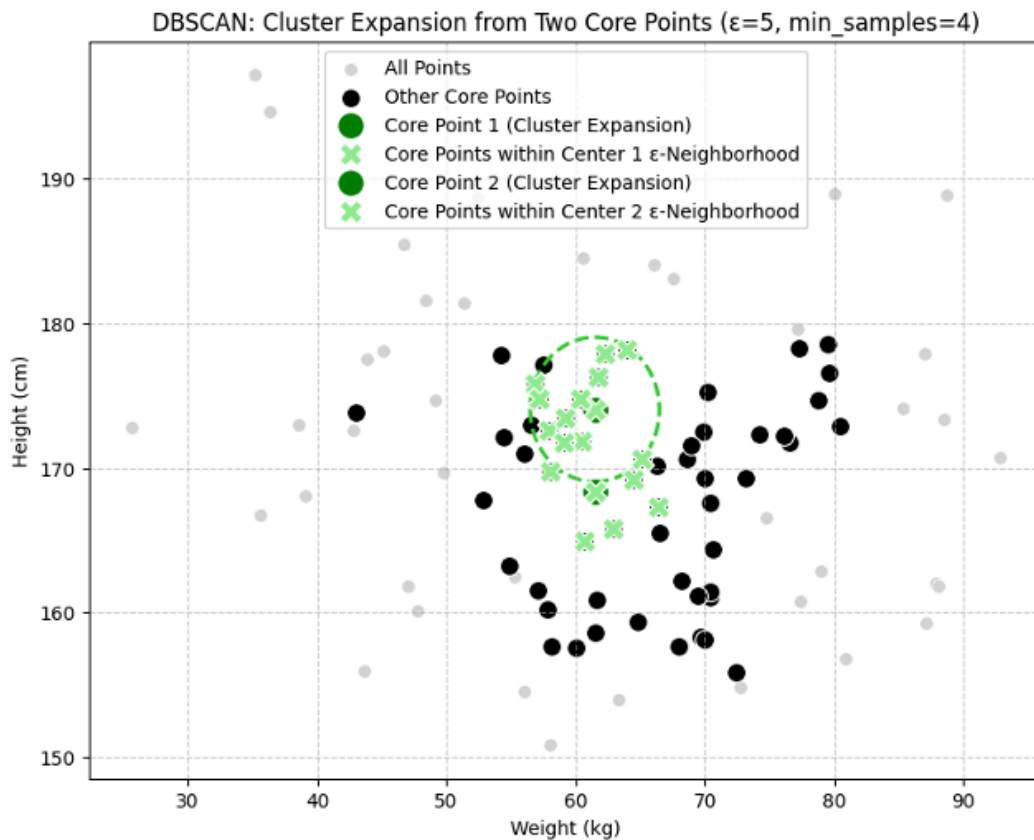
- Pick an unvisited core point. This becomes the **seed** of a new cluster.
- At this moment, the cluster is just one point.



The Core Point that overlaps the first circle are all added to the first cluster (the radius of the circle is the same)

- **Step 2: Expand Through Overlap**

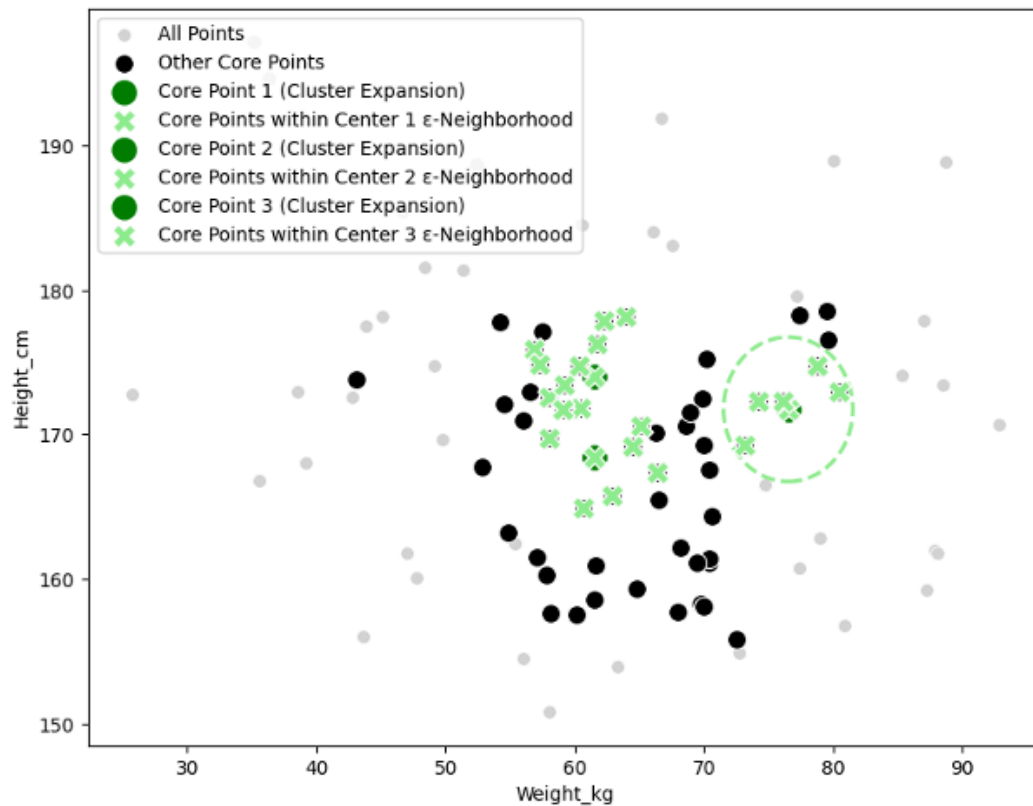
- Now look at its ϵ -circle. Any other **core point** whose circle overlaps with this one gets pulled into the same cluster.
- Because remember: Same ϵ radius $\rightarrow$ Overlapping neighborhoods $\rightarrow$ This means they belong to the same dense region.



Then the core points close to the growing cluster join it.

- **Step 3: Grow the Cluster**

- Now repeat the process. Each newly added core point brings its own neighborhood. And from there, the cluster starts to **expand outward**.
- Core points connect to nearby core points. Those connect to even more core points. And slowly, a dense region forms
- Like a chain reaction. Or like a ripple spreading through water.

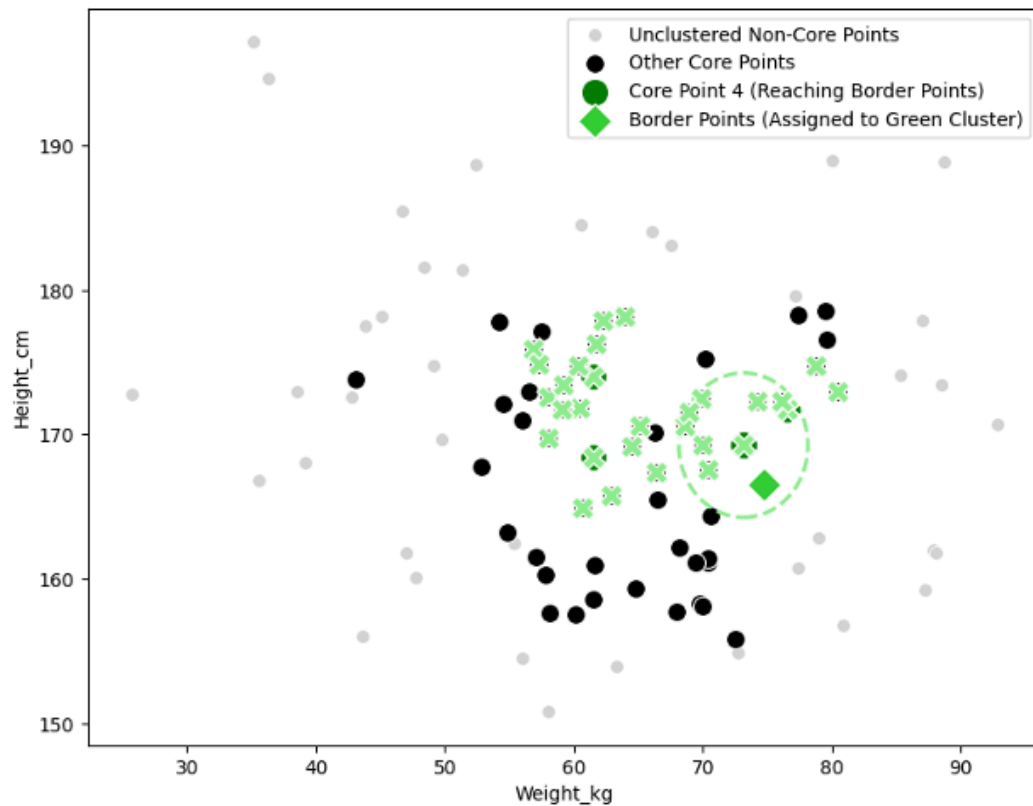


Extend it to close by Core Points

At this stage, something interesting happens. Even if you stop here, the **shape of the cluster is already defined**. Because core points define the structure.

- **Step 4: Add Non-Core Points**

- Only now, we look at the non-core points.
- For each one, we ask: "Is it within ϵ distance of at least one core point?"
 - If **yes** → it becomes a **border point** → added to the cluster
 - If **no** → it remains outside → labeled as **noise**



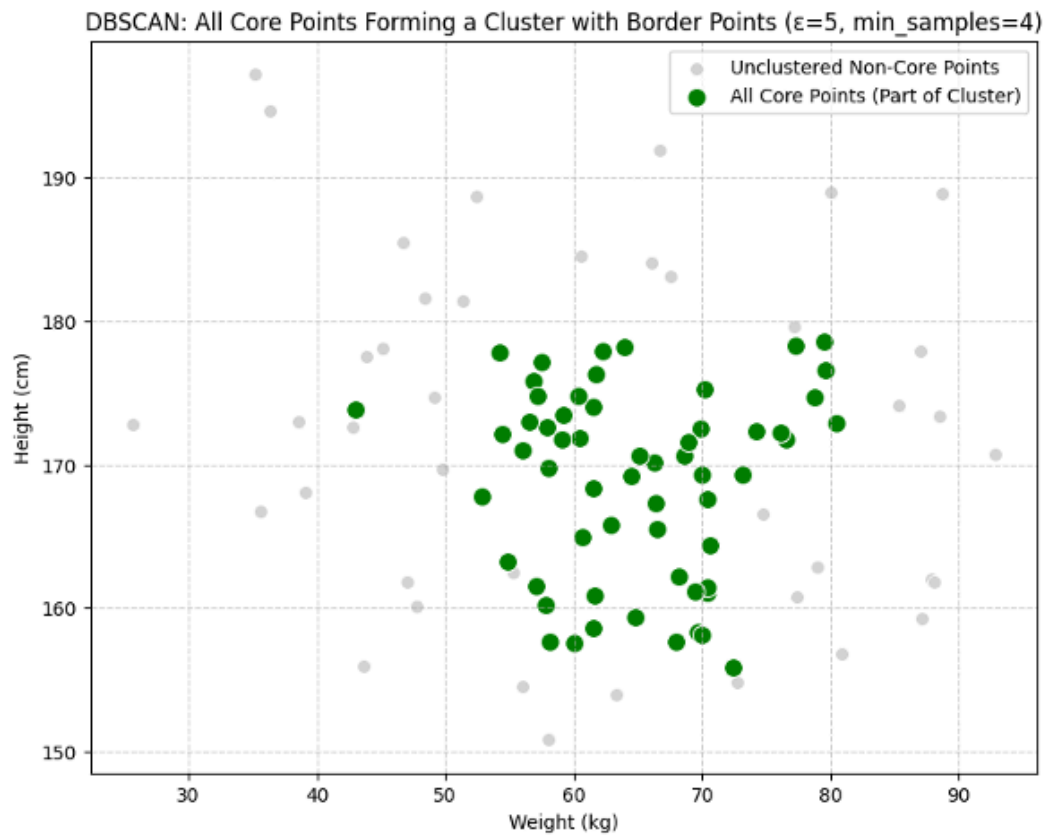
Eventually, the Non-Core Points are included in the cluster.

And here's a subtle but important case: If a non-core point is only close to **other non-core points**. That's not enough.

Because non-core points do not define clusters. → Only **core points** do.

So even if it feels "close to something". If that "something" isn't a core point. It still becomes **noise**.

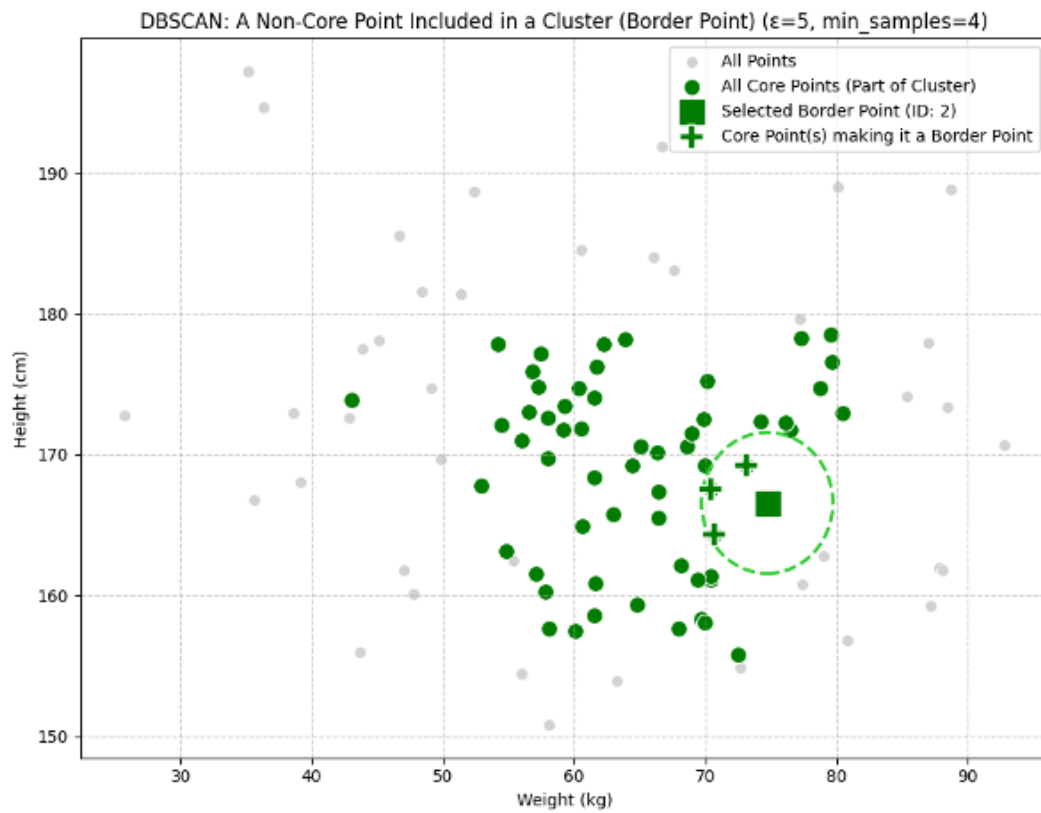
- **Finalizing a Cluster** - Once:
 - All reachable **core points** are included
 - All valid **border points** are added
- The cluster is complete. And we are done.



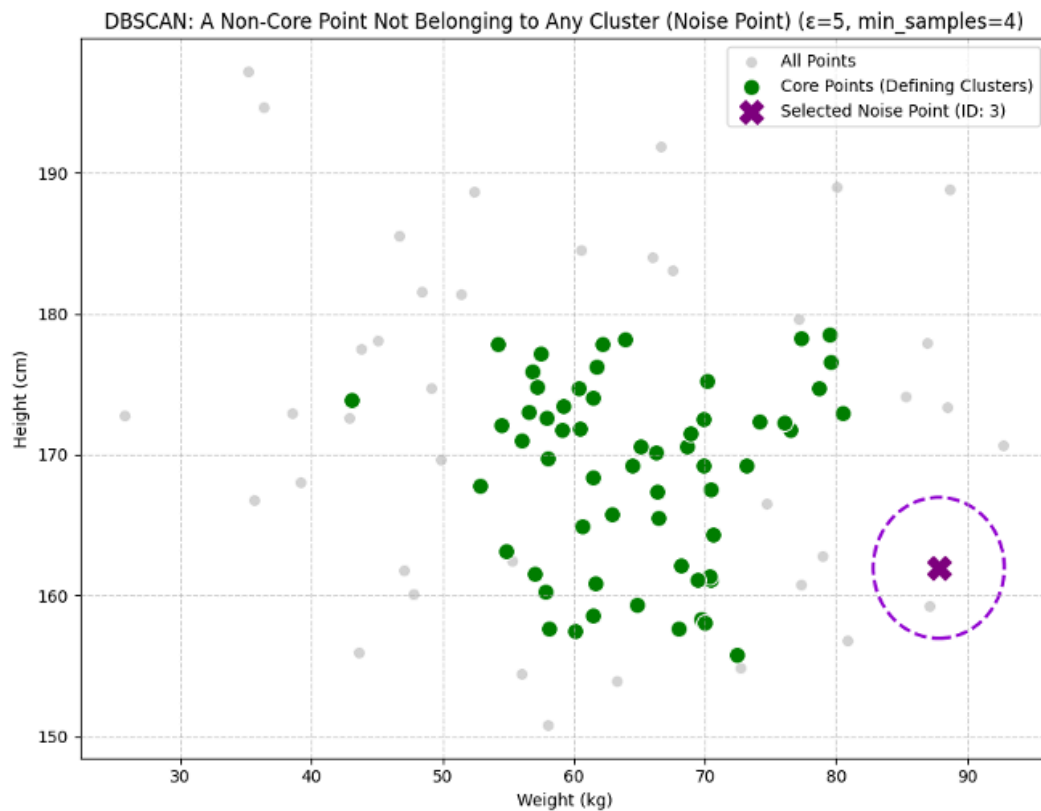
Initially, we include all Core Points to the first cluster

- **Moving Forward**

- DBSCAN then moves on to find another unvisited point:
 - If it's a core point → starts a new cluster
 - If not → checks connectivity
 - Otherwise → labels it as noise
- And this continues. Until every point is either part of a cluster or marked as noise.



Include a Non-Core Point to the first cluster if its circle overlaps at least one Core Point → Border Point



This non-core point here only overlaps another Non-Core Point, even if it is included in the first cluster, not a Core Point, hence not a part of this cluster → Noise Point

- **The Final Result** - Instead of forcing a fixed number of clusters. DBSCAN naturally produces:
 - Multiple clusters (however many the data supports)
 - With irregular shapes
 - And clearly separated noise points

Exactly how we would intuitively group the data.



Final DBSCAN result with three clusters

A Practical Note: Feature Scaling

There's one last detail, and it matters a lot. It might not look like it, but DBSCAN still relies on **distance** (usually Euclidean distance). So if your features are on different scales. Height in **centimeters** and Age in **years**.

Then distance calculations become biased. One feature dominates the other.

So before applying DBSCAN. You should **scale or standardize your data**. This ensures:

- Distances are meaningful
- ϵ is chosen properly
- Clusters are not artificially merged

Because in the end, DBSCAN is simple in concept. But its behavior depends entirely on how we define: **distance and density**.

5. DBSCAN with Python

One of the most interesting things about DBSCAN (especially when implementing it with Scikit-Learn) is what it **doesn't ask you to do**. It never

asks: "How many clusters do you want?"

And that's a big deal. Instead of forcing you to decide the number of clusters beforehand (like in K-Means), DBSCAN lets the data answer that question on its own.

How? Through two simple parameters:

- ϵ (**epsilon**) → how far we look for neighbors
- **min_samples (MinPts)** → how many neighbors are enough to call something "dense"

Together, these two define what **density** means. And once density is defined, clusters don't need to be specified. They simply **emerge**.

- Regions that are dense enough → become clusters
- Regions that are too sparse → become noise

So instead of saying: "Give me 3 clusters." You're saying something deeper: *"Show me where the data is truly dense... and ignore the rest."*

And that's the beauty of DBSCAN. It doesn't impose structure. It **discovers it**.

We are using the following dataset:

[dbscan_dataset.csv](#)

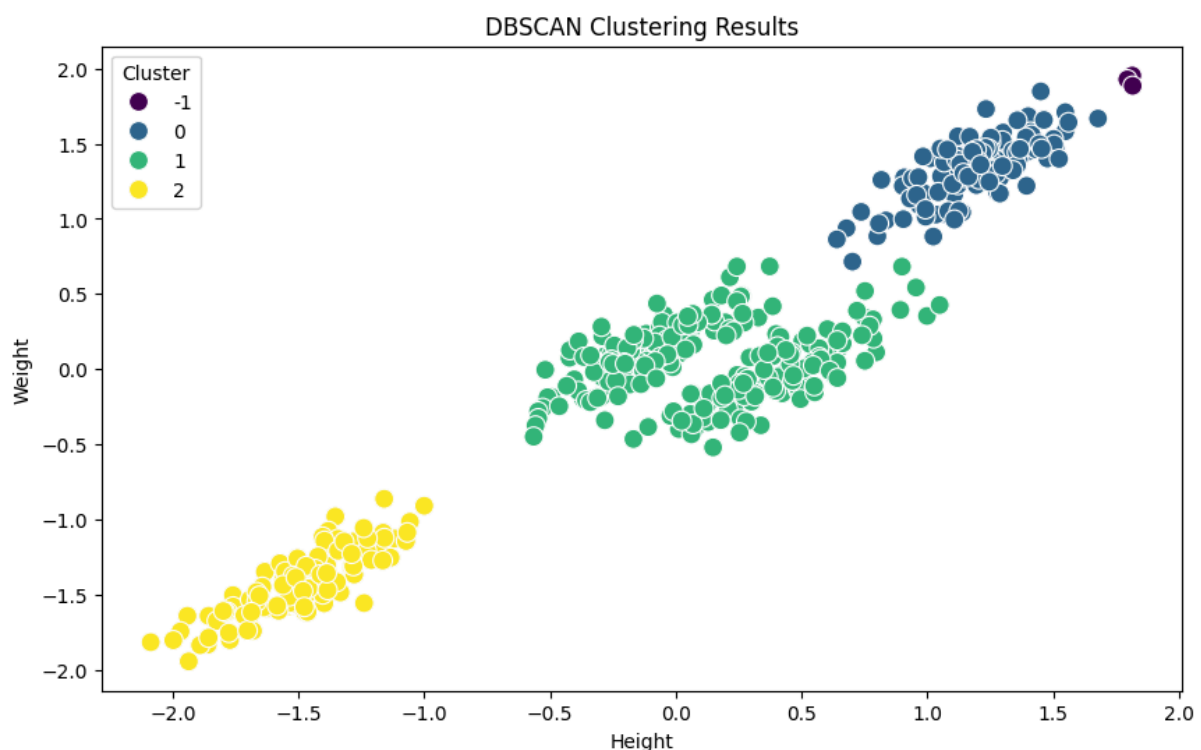
```
# DBSCAN with dataset
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.cluster import DBSCAN
from sklearn.preprocessing import StandardScaler

input_data = "dbscan_dataset.csv"
df = pd.read_csv(input_data)
# Standard Scaler
scaler = StandardScaler()
X = scaler.fit_transform(df[['Weight', 'Height']])
```



```
# Apply DBSCAN (Note: Adjust Epsilon to reflect the scaler)
model = DBSCAN(eps=0.2, min_samples=5)
model.fit(X)
labels = model.fit_predict(X)
# Extract cluster labels
DBSCAN_dataset = pd.DataFrame(X, columns=['Weight_scaled',
'Height_scaled'])
DBSCAN_dataset['Cluster'] = labels
```

```
# Plot the clusters
plt.figure(figsize=(10, 6))
sns.scatterplot(data=DBSCAN_dataset, x="Weight_scaled", y=
"Height_scaled",
hue="Cluster", palette="viridis", s=100)
plt.xlabel("Height")
plt.ylabel("Weight")
plt.title("DBSCAN Clustering Results")
plt.legend(title="Cluster")
plt.show()
```



Summary

DBSCAN, though we have to use an acronym to call it, is actually built on a very simple and intuitive idea: Clusters are not defined by centers. They are defined by **density**.

*Instead of forcing data into a fixed number of groups like K-Means, DBSCAN lets clusters **emerge naturally** based on how closely points are packed together.*

- Dense regions → become clusters
- Sparse regions → separate clusters
- Isolated points → labeled as noise

At the core of DBSCAN are just two parameters:

- ϵ (**epsilon**) → how far we look for neighbors
- *MinPts* → how many neighbors define a dense region

From these, everything else follows.

DBSCAN identifies three types of points:

- **Core points** → strong, dense regions that form clusters
- **Border points** → near clusters, but not dense enough themselves
- **Noise points** → isolated, not belonging anywhere

And from there, clusters grow organically:

Core points connect to other core points → Clusters expand through density →
Border points attach where appropriate → Noise is left behind.

What makes DBSCAN powerful is not just what it finds. But what it refuses to force.

- No need to specify the number of clusters
- No assumption of shape (clusters can be irregular, curved, nested)
- No obligation to assign every point

So while K-Means tries to **organize** the data, DBSCAN tries to **understand** it. And in real-world data—messy, noisy, imperfect—that difference matters.

References

<https://www.geeksforgeeks.org/machine-learning/dbscan-clustering-in-ml-density-based-clustering/>

<https://www.datacamp.com/tutorial/dbscan-clustering-algorithm>

<https://builtin.com/articles/dbscan>

<https://www.youtube.com/watch?v=RDZUdRSDOok>

Next Section:

 [17. Hierarchical Clustering](#)